

Documentation technique

Réflexions initiales technologiques sur le sujet.

Vite & Gourmand est une application web full-stack composée de deux parties distinctes : un frontend React qui gère tout ce que l'utilisateur voit, et un backend Symfony qui expose les données via une API. Ces deux parties communiquent entre elles exclusivement par des requêtes HTTP en JSON, sans que le backend génère la moindre page HTML.

J'ai utilisé **React** pour le frontend parce qu'il permet de découper l'interface en composants réutilisables, ce qui rend le code beaucoup plus facile à organiser quand le projet grandit. C'est exactement le cas de Vite & Gourmand, qui comporte plusieurs pages, un header et un footer présents sur toutes les pages, et des composants partagés. React permet aussi d'utiliser JSX pour écrire la structure des pages, React Router pour la navigation sans rechargement, et le Context API pour partager des données globales entre les composants, ce qui évite de devoir passer les informations de composant en composant via les props.

Tout le style de l'application est développé en **SCSS** parce qu'il me semble suffisamment complet : c'est la logique de programmation appliquée au CSS. Je le préfère au CSS classique parce qu'il permet de créer des variables de couleurs, de tailles et de polices, ainsi que des mixins qui sont des blocs de code réutilisables notamment pour le responsive design. Chaque page a son propre fichier .scss isolé grâce aux CSS Modules, ce qui évite les conflits de noms entre les styles des différentes pages.

Pour le backend, j'ai choisi **Symfony** parce que c'est le framework PHP de référence en France utilisé par une grande communauté. Symfony simplifie énormément les tâches : il impose une architecture claire et intègre de nombreux outils comme la gestion de la sécurité, le routing, l'injection de dépendances, l'envoi d'emails et la gestion des événements. Avec Doctrine ORM, il élimine aussi la nécessité d'écrire manuellement des requêtes SQL en permettant de manipuler les données via des objets PHP.

Pour exposer les données en API, j'ai utilisé **API Platform 3**, qui est une surcouche à Symfony. Il génère automatiquement une API REST à partir des entités **Doctrine**. Pour les routes simples comme la liste des menus ou des plats, cela m'a évité d'écrire des contrôleurs à la main. Pour les routes plus complexes comme création de commande, gestion du stock, calcul de prix, j'ai codé mes propres contrôleurs Symfony.

MySQL a été choisi pour gérer toutes les données structurées : utilisateurs, commandes, menus, plats, rôles. Doctrine ORM permet de manipuler ces données en PHP sous forme d'objets sans écrire de SQL à la main dans la plupart des cas, et les migrations Doctrine permettent de faire évoluer le schéma de façon organisée tout en gardant une trace de chaque changement.

En parallèle, **MongoDB** a été mis en place pour stocker trois types de données qui n'ont pas de structure fixe ou qui évoluent souvent : les avis clients, les statistiques des menus et les messages de contact. D'après moi, une base NoSQL est plus adaptée que MySQL pour ce genre de données, parce qu'on n'a pas besoin de définir un schéma rigide à l'avance. J'ai créé la base de données localement dans MongoDB Compass, puis j'ai migré vers MongoDB Atlas (la version hébergée dans le cloud) pour le déploiement en production.

Pour l'authentification, j'ai utilisé des **tokens JWT** signés avec une paire de clés RSA grâce au bundle **Lexik JWT** pour Symfony. Quand un utilisateur se connecte, le backend lui retourne un token qu'il doit ensuite envoyer dans chaque requête, c'est ce qui prouve son identité. Cette méthode ne nécessite pas de

maintenir une session côté serveur, ce qui est bien adapté à une API consommée par une application React. Les tokens expirent après 8 heures, après quoi l'utilisateur doit se reconnecter.

Pour les emails automatiques, j'ai utilisé le composant **Mailer de Symfony** avec des templates **Twig**. Pendant le développement en local, j'ai utilisé **Ethereal Mail**, un service qui intercepte les emails sans les envoyer vraiment, ce qui me permettait de vérifier que les templates s'affichaient correctement sans envoyer de vrais messages à de vraies adresses. Pour la production, j'ai basculé sur **Brevo** comme transporteur SMTP. Brevo propose un plan gratuit à 300 emails par jour et ne nécessite pas de domaine vérifié pour commencer.

Au cours du développement, j'ai eu recours à un **agent IA (GitHub Copilot)** pour m'assister sur certaines parties du code particulièrement complexes à implémenter seul. L'agent IA n'a pas remplacé mon travail de développeur : j'ai conçu l'architecture, choisi les technologies, structuré les entités, défini les routes et les règles métier. En revanche, pour des fonctionnalités où la logique technique dépassait mes connaissances actuelles, comme l'implémentation de la formule de Haversine pour le calcul des frais de livraison, ou certaines configurations avancées de JWT, j'ai utilisé l'agent IA en lui fournissant un contexte précis : les contraintes du projet, les fichiers concernés, le comportement attendu. Ce mode de travail m'a permis d'obtenir un code de qualité professionnelle sur ces points spécifiques, tout en comprenant ce que ce code faisait, puisque j'ai dû le lire, le tester, l'intégrer et le corriger quand il ne correspondait pas exactement au besoin. Je considère l'utilisation d'un agent IA comme un outil au même titre qu'une documentation officielle ou Stack Overflow.

Configuration de l'environnement de travail.

Le système d'exploitation utilisé pour ce projet est **Windows**. Ce choix s'explique par sa large compatibilité avec de nombreux outils de développement, notamment **WAMP** pour la gestion des serveurs web en local. Il offre une interface conviviale, un support étendu pour la plupart des logiciels courants du développement web, et répond bien à mes besoins personnels et professionnels.

Mon éditeur de code est **Visual Studio Code**. C'est l'IDE le plus répandu dans le développement web aujourd'hui : il est gratuit, léger, et dispose d'un grand nombre d'extensions utiles. J'ai notamment installé PHP Intelephense pour l'autocomplétion PHP, ESLint pour la vérification du code JavaScript, Prettier pour le formatage automatique et GitLens pour le suivi des modifications Git. Avoir le frontend et le backend dans le même espace de travail m'a permis de naviguer facilement entre les deux projets sans changer d'application.

Pour faire tourner le backend Symfony en local, j'ai utilisé WAMP64, qui installe Apache, MySQL et PHP sur Windows en une seule fois. J'ai configuré un virtual host Apache pour que le backend soit accessible à l'adresse `http://vitegourmand.local`.

Cela permet de simuler un vrai nom de domaine en développement et d'éviter des problèmes de chemin avec Symfony. La version de PHP configurée est PHP 8.2 qui est la version minimale requise par Symfony 7, avec les extensions nécessaires activées : `pdo_mysql` pour MySQL, `mongodb` pour MongoDB, `openssl` pour les clés JWT et `intl` pour Symfony.

Pour créer la base de données relationnelle, j'ai écrit toutes les requêtes SQL dans un fichier `database.sql` : création des tables, clés étrangères, données de démonstration. J'ai exécuté les requêtes depuis l'Invite de commandes Windows, une table à la fois — en copiant chaque bloc `CREATE TABLE` et en l'exécutant séparément pour pouvoir vérifier le résultat à chaque étape et corriger les erreurs au fur et à mesure. Cette façon de faire m'a aussi permis de mieux comprendre ce que je construisais, plutôt que de tout exécuter d'un coup. Une fois toutes les tables créées, j'ai géré les données via phpMyAdmin inclus dans

WAMP pour visualiser les enregistrements et faire des corrections sans écrire de SQL à la main. Les évolutions de schéma suivantes ont été gérées avec les migrations **Doctrine** via la commande `php bin/console doctrine:migrations:migrate`.

Pour la base de données MongoDB, j'ai utilisé **MongoDB Compass**, l'application de bureau officielle. Elle permet de se connecter au cluster **Atlas**, de parcourir les documents, de vérifier que les insertions fonctionnent correctement et de supprimer des données de test facilement, c'est l'équivalent de phpMyAdmin mais pour MongoDB. J'ai tout créé et testé dans Compass localement, avant de migrer vers MongoDB Atlas pour le déploiement en production.

Tout le code a été versionné avec **Git** et hébergé sur **GitHub** dans deux dépôts séparés : un pour le frontend, un pour le backend. J'ai appliqué une convention de branches simple : `main` pour la production, `develop` pour l'intégration, `release/prod` pour le déploiement et des branches `feature/` pour chaque nouvelle fonctionnalité. Les commits ont été faits régulièrement pour garder un historique lisible.

Pendant tout le développement, j'ai travaillé avec deux terminaux ouverts en permanence dans VS Code. Le premier était l'**Invite de commandes Windows (cmd)**, utilisé exclusivement pour lancer le serveur de développement React avec `npm start`. Ce terminal restait ouvert en permanence. Le deuxième était **PowerShell**, dédié à toutes les opérations Git : créer des branches, faire des commits, pousser le code sur GitHub. Séparer les deux évitait d'interrompre le serveur React à chaque commande Git. Dans de très rares occasions j'ai utilisé le terminal intégré de VS Code directement, j'avais l'impression qu'il provoquait des ralentissements importants.

Pour les dépendances PHP, j'ai utilisé **Composer**, le gestionnaire de paquets standard de l'écosystème PHP. Il m'a permis d'installer **Symfony 7.4**, **API Platform**, **Doctrine ORM**, **Lexik JWT Bundle**, **NelmioCorsBundle**, **Symfony Mailer** et toutes les autres bibliothèques listées dans `composer.json`. Le dossier `vendor/` généré par Composer n'est pas commité sur GitHub. Pour le frontend React, j'ai utilisé **Node.js** et **npm**. La commande `npm install` installe toutes les dépendances, et `npm start` lance un serveur local sur `http://localhost:3000` qui recharge automatiquement la page à chaque modification de fichier. Les principales bibliothèques installées sont **React Router v7** pour la navigation, **Axios** pour les appels API, **Formik** et **Yup** pour la gestion et la validation des formulaires, **Recharts** pour les graphiques des statistiques admin et **Framer Motion** pour les animations.

Pour tester les emails sans en envoyer de vrais, j'ai utilisé **Ethereal Mail** pendant tout le développement. C'est un service en ligne qui crée une fausse boîte mail : les emails sont interceptés et affichés dans une interface web sans jamais être délivrés. Il suffit de récupérer les identifiants SMTP fournis sur le site Ethereal et de les renseigner dans le fichier `.env.local` du backend. Une fois satisfait du résultat, j'ai basculé sur **Brevo** pour la production.

Pour déboguer les échanges entre le frontend et le backend, j'utilisais les outils de développement du navigateur en permanence. L'onglet **Console** affichait en temps réel les erreurs JavaScript, les avertissements React et les problèmes de réseau. L'onglet **Network** me permettait de voir les requêtes envoyées, les réponses reçues et les codes HTTP retournés par le backend. Pour les endpoints backend en GET, j'entrais directement l'URL dans la barre d'adresse du navigateur — par exemple `http://vitegourmand.local/api/menus`, ce qui affiche la réponse JSON complète et permet de vérifier rapidement que les données sont bien formatées, sans avoir besoin d'un outil externe comme Postman.

Déploiement de l'application et les différentes étapes.

Avant de commencer à déployer quoi que ce soit, j'avais deux projets à mettre en ligne séparément : le frontend React et le backend Symfony. Comme ils sont complètement indépendants l'un de l'autre, j'ai dû

trouver deux hébergeurs différents adaptés à chaque technologie. L'objectif était que les deux soient accessibles en HTTPS sur de vraies URLs, que le frontend puisse communiquer avec le backend, et que les bases de données soient accessibles depuis le serveur de production.

Avant de connecter les hébergeurs à la branche main, tout le développement se faisait sur des branches feature/ mergées dans la branch dev. Quand j'ai estimé que l'application était prête à être déployée, j'ai créé une branche release/prod à partir de dev.

C'est sur cette branche que j'ai effectué les derniers ajustements spécifiques à la production : la configuration des variables d'environnement, les URLs de production dans le code, les derniers tests. Ça m'a permis de corriger des problèmes qui n'apparaissaient qu'en conditions réelles sans toucher à dev ni à main. Une fois que tout fonctionnait correctement en production depuis cette branche, j'ai fait le merge final dans main. Depuis ce moment-là, main représente l'état stable et déployé de l'application, et c'est la branche connectée à Vercel et Clever Cloud pour les déploiements automatiques.

Déploiement du frontend sur Vercel

Pour le frontend React, j'ai choisi Vercel. C'est une des plateformes de déploiement les plus populaires pour les applications React. La première étape a été de pousser le code du frontend sur l'interface de Vercel, j'ai importé ce dépôt directement en me connectant avec mon compte GitHub. Vercel détecte automatiquement que c'est un projet Create React App et configure le build.

Un point important était la gestion de la navigation côté client avec React Router. En production, quand un utilisateur accède directement à une URL comme `https://vitegourmand-frontend.vercel.app/menu/list/`, Vercel cherche un fichier correspondant sur le serveur et ne trouve rien — ce qui retourne une erreur 404. Pour résoudre ça, j'ai ajouté un fichier `vercel.json` à la racine du projet frontend qui redirige toutes les requêtes vers `index.html`, laissant React Router gérer la navigation côté client.

Depuis ce moment-là, le déploiement est entièrement automatique : chaque fois que je fais un git push origin main sur le dépôt frontend, Vercel déclenche automatiquement un nouveau build et met l'application en ligne en quelques minutes. C'est très pratique pendant le développement.

Déploiement du backend sur Clever Cloud

Pour le backend Symfony, j'ai décidé d'utiliser Clever Cloud. Clever Cloud propose un hébergement PHP avec MySQL intégré, et leur documentation pour Symfony est claire.

Sur Clever Cloud, j'ai créé une application de type PHP et j'ai connecté le dépôt GitHub du backend. Clever Cloud détecte Symfony et sait comment le déployer. J'ai dû configurer quelques fichiers pour que le déploiement fonctionne correctement. Comme pour Vercel, une fois la connexion GitHub établie, chaque push sur la branche main déclenche automatiquement un redéploiement du backend.

Une étape importante côté Symfony était de s'assurer que l'application tourne bien en mode prod et non en mode dev. J'ai dû m'assurer que la variable d'environnement `APP_ENV=prod` était bien définie sur le serveur.

Configuration de la base de données MySQL sur Clever Cloud

Clever Cloud propose des add-ons directement dans son interface. J'ai ajouté un add-on MySQL à mon application backend. Une fois l'add-on créé, Clever Cloud fournit automatiquement une variable d'environnement `DATABASE_URL` avec toutes les informations de connexion (hôte, port, nom de la base, identifiants). Il suffit de référencer cette variable dans le fichier `.env` de Symfony.

Pour créer les tables en production, j'ai utilisé les migrations Doctrine. Une fois le backend déployé, j'ai exécuté la commande `php bin/console doctrine:migrations:migrate --no-interaction` depuis la console de

Clever Cloud pour appliquer toutes les migrations et créer le schéma de la base de données. Clever Cloud fournit aussi une interface phpMyAdmin accessible depuis le dashboard, ce qui m'a permis de vérifier que les tables avaient bien été créées et d'insérer les données de démonstration.

Configuration de MongoDB Atlas

Pour MongoDB, j'avais déjà créé ma base de données localement avec MongoDB Compass pendant le développement. Pour la production, j'ai utilisé MongoDB Atlas, qui est le service cloud officiel de MongoDB. J'ai créé un cluster gratuit de type Free M0, qui est suffisant pour ce projet.

Dans Atlas, j'ai créé un utilisateur de base de données avec un identifiant et un mot de passe, puis j'ai autorisé les connexions depuis n'importe quelle adresse IP (0.0.0.0/0) pour que le serveur Clever Cloud puisse se connecter. Atlas fournit ensuite une URI de connexion du type `mongodb+srv://...@...mongodb.net` que j'ai renseignée dans les variables d'environnement du backend. J'ai ensuite recréé manuellement les trois collections sur Atlas (reviews, menu_stats, contact_messages) et importé les quelques données de test que j'avais en local.

Configuration des variables d'environnement

C'est une étape cruciale : il ne faut jamais committer des informations sensibles comme des mots de passe ou des clés secrètes dans le code sur GitHub. Toutes ces informations sont stockées comme variables d'environnement directement sur Clever Cloud, dans la section "Variables d'environnement" du dashboard.

J'y ai renseigné :

- APP_ENV
- APP_SECRET
- DATABASE_URL,
- MONGODB_URI
- MONGODB_DB
- MAILER_DSN avec les identifiants Brevo
- JWT_PASSPHRASE pour déchiffrer les clés RSA
- CORS_ALLOW_ORIGIN avec l'URL du frontend Vercel

Le fichier .env.local du développement local n'est jamais comité, il est dans le .gitignore.

Pour les clés RSA du JWT (private.pem et public.pem), elles sont stockées dans le dossier config/jwt/ du projet et référencées dans `lexik_jwt_authentication.yaml`.

Ces fichiers sont dans le .gitignore en local, mais sur Clever Cloud j'ai dû les passer en variables d'environnement.

Configuration des emails avec Brevo

Pour les emails en production, j'ai créé un compte sur Brevo et j'ai vérifié l'adresse email expéditrice. Brevo fournit des identifiants SMTP : un login (au format adresse email Brevo), un mot de passe SMTP et le serveur `smtp-relay.brevo.com` sur le port 587.

J'ai renseigné ces informations dans la variable MAILER_DSN sur Clever Cloud.

Vérification finale et mise en ligne

Une fois tout configuré, j'ai vérifié que les deux applications communiquaient correctement en testant les principaux parcours depuis le navigateur : connexion, inscription, parcours de commande, envoi d'un message de contact.

J'ai aussi vérifié dans les DevTools du navigateur que les requêtes portaient bien vers la bonne URL de

production et que les headers CORS étaient correctement présents dans les réponses. L'application est aujourd'hui accessible à l'adresse <https://vitegourmand-frontend.vercel.app>.